

자료구조 실습

09/23

오늘 실습 순서

- 코드 구조 확인 — UnsortedList의 data · length, 오늘 만들 5개 연산
- 문제 1~5: 문제를 먼저 보고 직접 작성한 뒤, 같이 풀이 확인
 - 1) appendItem 2) insertItem 3) improved_insertItem
 - 4) findItem 5) removeItem
- 문제 6: 시간복잡도 체감 실험 — N을 2배씩 키우며 직접 측정
- 연산별 Big-O 총정리

실습 진행 방법

- practice.py를 열고 문제 1~5의 NEED_TO_SOLVE를 채움
- NEED_TO_SOLVE는 아무 동작도 하지 않는 빈 자리 -> 실행은 되지만 결과가 틀림
- 실행하면 체크포인트가 PASS / FAIL로 나옴. 전부 PASS면 문제 6 실험이 이어서 실행됨

코드 받기

```
git clone https://github.com/msforgood/data-structure-assist.git
```

```
# 지난주에 이미 받았다면  
git pull origin main
```

실행

```
cd 2026-가을/w4-0923  
python3 practice.py
```

```
# 정답 코드: practice_answer.py
```

규칙

AI 사용 금지.
파이썬 list의 append / insert / remove / index / in 사용 금지
-> self.data 배열과 self.length만 직접 조작

이번 주에 다룰 클래스 — UnsortedList

- 배열 크기는 max_size로 고정, 실제 원소 개수는 length가 따로 기억
- data[0] ~ data[length-1]만 의미 있는 값, 그 뒤는 쓰레기 값

이미 작성되어 있는 부분

```
def __init__(self, max_size=MAX_SIZE):
    self.data = [None] * max_size    # 저장 공간 (크기 고정)
    self.length = 0                 # 지금 들어 있는 원소 개수

def size(self):    return self.length
def isFull(self):  return self.length == self.max_size
def isEmpty(self): return self.length == 0
def getItem(self, pos): # pos 범위 검사 후 self.data[pos] 반환
def clear(self):    self.length = 0
def toList(self):   # 확인용 - data[0..length-1]을 파이썬 리스트로 복사
```

주어진 것 : size, isFull, isEmpty, getItem, clear, toList

채울 것 : appendItem, insertItem, improved_insertItem, findItem, removeItem (문제 1~5)

오늘 만들 5개 연산 — 무엇이 다른가

- 전부 '값 하나를 넣거나 · 찾거나 · 지우는' 연산 -> 차이는 그 과정에서 하는 일의 양

문제 1~5에서 채울 연산

연산	하는 일	다른 점
<code>appendItem()</code>	맨 뒤에 붙임	위치를 고를 수 없음. 빈 자리에 쓰기만 함
<code>insertItem()</code>	pos 자리에 끼워 넣음	위치를 고르는 대신 pos 뒤 원소를 전부 한 칸씩 밀어야 함
<code>improved_insertItem()</code>	pos 자리에 끼워 넣음	밀지 않음. 원래 값을 맨 뒤로 보내므로 순서가 바뀜
<code>findItem()</code>	값의 위치를 찾음	위치를 모르는 상태에서 시작 -> 0번부터 훑어야 함
<code>removeItem()</code>	값을 찾아 지움	findItem으로 찾은 뒤 맨 뒤 값으로 빈칸을 메움

차이를 만드는 축은 둘뿐

- ① 위치를 아는가(`append · insert`) / 찾아야 하는가(`find · remove`)
- ② 순서를 지키는가(`insert`) / 안 지켜도 되는가(`improved_insert`)

계산 횟수와 Big-O는 문제 1~6에서 직접 세어 확인

파이썬 list의 기능을 쓰지 않는 이유

- data.append(value) 한 줄이면 끝나지만, 그 안에서 몇 번 움직였는지는 보이지 않음
- 이번 주에 세는 것은 '계산 횟수' -> 인덱스를 직접 다뤄야 횟수가 보임

사용 금지

```
lst.append(v)
lst.insert(i, v)
lst.remove(v)
lst.index(v)
v in lst
```

이렇게 씀

```
self.data[i] = value
self.length += 1
while i > pos:
    ...
    i -= 1
```

self.data 자체는 파이썬 리스트지만, 여기서는 '크기가 고정된 배열'로만 씀
인덱스로 읽고 쓰는 것(data[i])만 허용. 크기를 바꾸는 기능은 사용하지 않음

appendItem() — 맨 뒤에 값 붙이기

완성할 코드 (practice.py)

```
def appendItem(self, value):  
    if self.isFull():  
        raise OverflowError("List is full")  
  
    # TODO: data의 빈 자리에 value를 저장하고, length를 1 늘리기  
    NEED_TO_SOLVE
```

- 지금 원소가 length개일 때, 다음에 쓸 빈 자리의 인덱스는 몇 번인가?
- 테스트: 빈 리스트에 25, 13, 47을 차례로 넣으면 toList()의 결과는?



먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

빈 자리에 쓰고 length를 1 늘림

- length는 '개수'이자 동시에 '다음 빈 자리의 인덱스' (0부터 세기 때문)

풀이 코드

```
def appendItem(self, value):
    if self.isFull():
        raise OverflowError("List is full")
    self.data[self.length] = value    # 1회
    self.length += 1                 # 1회
```

계산 횟수

저장 1회 + 증가 1회 = 항상 2회

length가 3이든 100만이든 2회로 같음
-> O(2) -> O(1)

실행 결과

[PASS] appendItem 3회: [25, 13, 47]

insertItem() — pos 자리에 끼워 넣기

완성할 코드 (practice.py)

```
def insertItem(self, pos, value):  
    # (isFull / pos 범위 검사는 이미 작성되어 있음)  
  
    # TODO: i를 length에서 시작해 pos보다 클 동안,  
    #       data[i]에 data[i-1]을 복사하고 i를 1씩 줄이기  
    NEED_TO_SOLVE  
  
    # TODO: 비워진 pos 자리에 value를 넣고, length를 1 늘리기  
    NEED_TO_SOLVE
```

- [25, 13, 47]에 insertItem(1, 7)을 하면 결과는?
- 앞에서부터(i = pos) 밀면 왜 안 되는지 직접 그려보기



먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

뒤에서부터 한 칸씩 밀고, 빈 자리에 쓰기

풀이 코드

```
i = self.length
while i > pos:
    self.data[i] = self.data[i - 1]
    i -= 1
self.data[pos] = value
self.length += 1
```

insertItem(1, 7) 단계별 변화

시작	[25, 13, 47]	length=3
i=3	data[3] <- data[2]	
	[25, 13, 47, 47]	
i=2	data[2] <- data[1]	
	[25, 13, 13, 47]	
i=1 -> 종료	(i > pos 가 거짓)	
pos 저장	[25, 7, 13, 47]	length=4

계산 횟수

밀기는 length - pos 회. 최악은 pos=0일 때 N회.
여기에 저장 1회 + 증가 1회 -> 최악 N+2회 -> O(N+2) -> O(N)
뒤에서부터 밀는 이유: 앞에서부터 밀면 아직 옮기지 않은 값을 덮어씀

실행 결과

[PASS] insertItem(1, 7): [25, 7, 13, 47]

improved_insertItem() — 밀지 않고 끼워 넣기

- 비정렬 리스트는 순서에 의미가 없음. 그렇다면 굳이 뒤를 다 밀어야 할까?

완성할 코드 (practice.py)

```
def improved_insertItem(self, pos, value):  
    # (isFull / pos 범위 검사는 이미 작성되어 있음)  
  
    # TODO: data[pos]에 있던 값을 맨 뒤(data[length])로 옮기기  
    NEED_TO_SOLVE  
  
    # TODO: pos 자리에 value를 쓰고, length를 1 늘리기  
    NEED_TO_SOLVE
```

- [25, 13, 47]에 improved_insertItem(1, 7)을 하면 결과는?
- 문제 2와 결과가 다름. 무엇이 어떻게 다른지 확인해보기



먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

원래 값을 맨 뒤로 보내고 그 자리에 쓰기

풀이 코드

```
self.data[self.length] = self.data[pos] # 1회
self.data[pos] = value # 1회
self.length += 1 # 1회
```

improved_insertItem(1, 7) 단계별 변화

시작	[25, 13, 47]	length=3
뒤로 복사	[25, 13, 47, 13]	
pos 저장	[25, 7, 47, 13]	length=4

계산 횟수

복사 1회 + 저장 1회 + 증가 1회 = 항상 3회 -> O(1)
length가 16000이어도 3회. 문제 2의 N+2회와 비교할 것.

주의 — 언제 쓸 수 있는가

쓸 수 있음 : 비정렬 리스트. 13이 어디로 가든 상관없음
쓸 수 없음 : 정렬 리스트. 13을 맨 뒤로 보내면 크기 순서가 깨짐 -> 반드시 밀어야 함

실행 결과

[PASS] improved_insertItem(1, 7): [25, 7, 47, 13]

findItem() — 값이 어디 있는지 찾기

- 정렬되어 있지 않으므로 0번부터 하나씩 비교하는 방법밖에 없음 (선형 탐색)
- 수업에서는 True / False를 반환했지만, 여기서는 위치를 반환 (문제 5에서 다시 씀)

완성할 코드 (practice.py)

```
def findItem(self, item):  
    pos = 0  
    # TODO: pos가 length보다 작을 동안 data[pos]와 item을 비교하고,  
    #       같으면 pos를 반환. 다르면 pos를 1 늘리기  
    NEED_TO_SOLVE  
    return -1
```

- 테스트: [25, 7, 47, 13]에서 findItem(47)과 findItem(99)의 반환값은?



먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

앞에서부터 하나씩 비교

풀이 코드

```
def findItem(self, item):  
    pos = 0  
    while pos < self.length:  
        if self.data[pos] == item:  
            return pos  
        pos += 1  
    return -1
```

계산 횟수

최선 : 첫 원소가 정답이면 1회
최악 : 마지막이거나 없으면 N회

Big-O는 최악 기준 -> $O(N)$

찾는 값이 없을 때가 가장 느림
(문제 6 실험 2에서 이 경우를 측정)

실행 결과

```
[PASS] findItem(47): 2      # [25, 7, 47, 13]의 2번 자리  
[PASS] findItem(99): -1    # 없으면 -1
```

removeItem() — 값을 찾아서 지우기

- ① findItem으로 위치 찾기 (없으면 False를 반환하고 끝)
- ② 맨 뒤 원소를 그 자리로 옮겨 빈칸 메우기 (순서는 신경 쓰지 않음)
- ③ length를 1 줄이기

완성할 코드 (practice.py)

```
def removeItem(self, value):  
    # TODO: 위 ① ② ③을 그대로 코드로 옮기기  
    NEED_TO_SOLVE  
    return False
```

- 테스트: [25, 7, 47, 13]에서 removeItem(7)을 하면 리스트는 어떤 모양?
- 지운 자리를 '맨 뒤 값'으로 메우면, 밀어서 메울 때와 무엇이 달라질까?



먼저 3~5분 동안 직접 작성 (풀이는 다음 장에서 확인)

찾고, 맨 뒤 값으로 메우고, length를 줄임

풀이 코드

```
def removeItem(self, value):
    pos = self.findItem(value)
    if pos == -1:
        return False
    self.data[pos] = self.data[self.length - 1]
    self.data[self.length - 1] = None
    self.length -= 1
    return True
```

removeItem(7) 단계별 변화

시작	[25, 7, 47, 13]	length=4
찾기	findItem(7) -> 1	
메우기	data[1] <- data[3]	
	[25, 13, 47]	length=3

계산 횟수

탐색 최악 N회 + 메우기 1회 + 감소 1회 -> O(N)
메우는 일만 O(1)로 줄일 수 있고, 탐색 비용은 피할 수 없음

실행 결과

[PASS] removeItem(7): (True, [25, 13, 47]) [PASS] removeItem(99): (False, [25, 7, 47, 13])

문제 1~5를 다 채우면 나오는 화면

python3 practice.py

```
=== 체크포인트 ===  
[PASS] appendItem 3회: [25, 13, 47]  
[PASS] insertItem(1, 7): [25, 7, 13, 47]  
[PASS] improved_insertItem(1, 7): [25, 7, 47, 13]  
[PASS] findItem(47): 2  
[PASS] findItem(99): -1  
[PASS] removeItem(7): (True, [25, 13, 47])  
[PASS] removeItem(99): (False, [25, 7, 47, 13])  
결과: 전부 통과
```

- FAIL이 남아 있으면 실험은 실행되지 않음 -> 먼저 전부 PASS로 만들기
- 막히면 practice_answer.py와 비교해서 어느 줄이 다른지 찾기

시간복잡도 체감 실험 — 무엇을 재는가

- 실험 1: 같은 '맨 앞(pos=0) 삽입'을 insertItem과 improved_insertItem으로 각각 측정
- 실험 2: 값을 찾아야 하는 findItem과 위치를 아는 getItem을 측정
- N을 1000 -> 2000 -> 4000 -> 8000 -> 16000으로 2배씩 키우며 반복
- improved는 1회가 수백 나노초라, 반복을 더 많이 해야 측정 오차에 묻히지 않음

이미 작성되어 있는 측정 코드

```
lst = build(n, repeat + 100)          # N개짜리 리스트 생성
start = time.perf_counter()
for _ in range(repeat):                # insertItem 200회 / improved 4000회
    f(0, -1)                            # 맨 앞에 삽입
elapsed = time.perf_counter() - start
return elapsed / repeat * 1_000_000    # 1회당 마이크로초(us)
```

결과를 보기 전에 예상해보기 — N이 2배가 되면 각 연산의 시간은 몇 배?

실험 1 — insertItem vs improved_insertItem

측정 결과 (pos=0 삽입 1회 평균, 실행 환경마다 값은 다름)

N	insertItem	improved	성장	배율
1000	50.0us	0.141us	1.0x	353x
2000	97.8us	0.132us	2.0x	741x
4000	189.2us	0.132us	3.8x	1433x
8000	379.3us	0.133us	7.6x	2859x
16000	774.0us	0.141us	15.5x	5478x

읽는 법

성장 : insertItem은 N이 16배면 시간도 약 15배 -> 계산 횟수가 N에 비례 -> $O(N)$
improved는 N이 16배가 되어도 0.13us 그대로 -> 항상 3회만 계산 -> $O(1)$
배율 : 두 방식의 시간 차이. N=1000에서 353배, N=16000에서 5478배 -> 배율 자체가 커짐

측정값은 컴퓨터마다 다름. 절대 시간이 아니라 'N이 2배일 때 몇 배가 되는가'를 보는 것

차이가 왜 이렇게까지 벌어지는가

- N은 삽입 횟수가 아니라 리스트에 이미 들어 있는 원소 개수 -> 아래 표는 삽입 1회 기준
- 실행 시간 = 계산 횟수 × 1회당 비용. 1회당 비용은 상수 -> 시간 배율 = 계산 횟수 배율
- pos=0 삽입 1회에서 insertItem은 N칸을 전부 한 칸씩 밀고, improved는 3번만 건드림

삽입 1회에 드는 계산 횟수

N (리스트 길이)	insertItem 1회 (pos=0)	improved 1회	계산 횟수 배율	측정 시간 배율
1000	1000번 밀기 + 2 = 1002회	3회	334x	353x
4000	4000번 밀기 + 2 = 4002회	3회	1334x	1433x
16000	16000번 밀기 + 2 = 16002회	3회	5334x	5478x

배율 = $(N + 2) / 3$ -> N에 비례해서 같이 커짐. N=16000이면 약 5334배, 측정도 그대로 따라감
 $O(N)$ vs $O(1)$ 은 '몇 배 느림'이 고정된 것이 아니라, N이 커질수록 배율이 함께 커진다는 뜻

N을 10배로 키우면 차이도 약 10배 더 벌어짐 — 이것이 $O(N)$ 과 $O(1)$ 의 실질적 의미

실험 2 — findItem vs getItem

측정 결과 (1회 평균, findItem은 없는 값을 찾는 최악의 경우)

N	findItem	getItem
1000	41.9us	0.30us
2000	84.9us	0.07us
4000	169.6us	0.07us
8000	337.1us	0.07us
16000	676.4us	0.07us

읽는 법

findItem : N이 2배 -> 시간도 2배. 끝까지 훑어야 하므로 $O(N)$
getItem : 인덱스로 한 번에 접근하므로 N과 무관 -> $O(1)$
정리: 값을 찾아야 하는 연산은 $O(N)$, 위치를 이미 아는 연산은 $O(1)$

다음 주 질문 — 정렬만 되어 있다면 이 탐색을 $O(\log N)$ 으로 줄일 수 있음

오늘 구현한 연산의 Big-O

연산	하는 일	계산 횟수	Big-O
<code>appendItem</code>	맨 뒤에 저장 + length 증가	2	$O(1)$
<code>insertItem</code>	pos 뒤를 전부 밀고 삽입	최악 $N+2$	$O(N)$
<code>improved_insertItem</code>	원래 값을 맨 뒤로 보내고 삽입	3	$O(1)$
<code>findItem</code>	0번부터 하나씩 비교 (선형 탐색)	최악 N	$O(N)$
<code>removeItem</code>	<code>findItem</code> 으로 찾고 맨 뒤 값으로 메움	최악 $N+2$	$O(N)$
<code>getItem / size / clear</code>	위치를 알고 접근하거나 length만 조작	1	$O(1)$

위치를 이미 아는 연산은 $O(1)$, 값을 찾아야 하는 연산은 $O(N)$.
순서를 지킬 의무가 없는 비정렬 리스트라서 `insertItem`을 $O(1)$ 로 바꿀 수 있었음

4주차 실습 끝

다음 시간 — 정렬 리스트와 이진 탐색